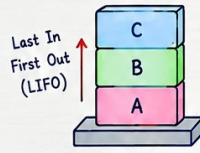


UNIT - II

Stacks and Queues

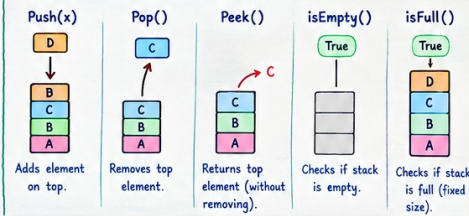
Design by
Sarafray Navaz

1. Stack - Basic Idea

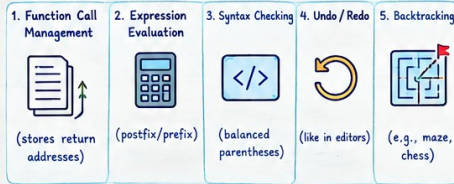


- Think of a stack like a pile of plates.
- You can only add or remove from the top.
- Last item comes out first.

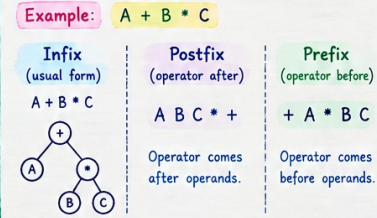
2. Primitive Operations on Stack



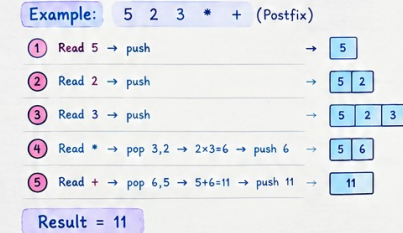
3. Stack Applications



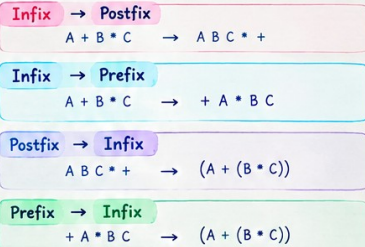
4. Infix, Postfix and Prefix Expression



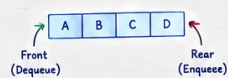
5. Evaluation of Postfix Expression



6. Conversion Between Expressions

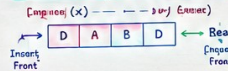


7. Queue - Basic Idea

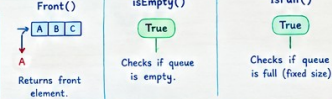
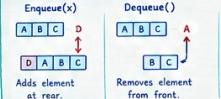


- Think of a queue like a line at a ticket counter.
- First in, First out (FIFO).
- You can add at the rear and remove from the front.

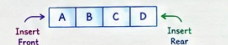
8. Primitive Operations on Queue



8. Primitive Operations on Queue



9. D-Queues (Double Ended Queue)



- Can insert and delete from both ends.
- Useful in deques, sliding window, etc.

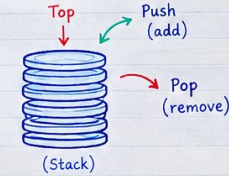
10. Priority Queue



- Elements are arranged based on priority.
- Higher priority elements are served first.
- Implemented using heap or balanced tree.

1 Stack - Basic Idea

- A stack is a linear data structure where insertion and deletion happen at the same end (top).
- It follows Last In First Out (LIFO) principle.
- Think of it as a pile of plates, where you can add or remove only from the top.



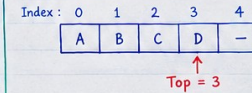
LIFO
(Last In
First Out)

2 Stack Implementation

(A) Array Implementation (Static)

Definition: In array implementation, the stack is represented using a fixed-size array. The top variable stores the index of the last element.

Representation:

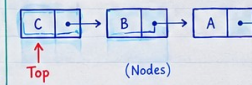


- Fixed size
- Cannot grow
- Overflow if full

(B) Dynamic Implementation (Using Linked List)

Definition: In dynamic implementation, the stack is represented using a linked list. Each node contains data and a pointer to the next node. The top points to the first node.

Representation:

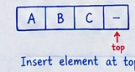


- No fixed size
- Grows / shrinks as needed
- Overflow / Underflow not an issue

3 Push, Pop and Peek Operations

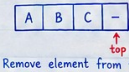
Push(x)

- Check if stack is full (in array).
- If not full, increment top and insert x.



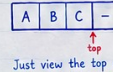
Pop()

- Check if stack is empty.
- If not empty, return top element and decrement top.



Peek()

- Check if stack is empty.
- Return the element at top (without removing).



4 Infix, Postfix and Prefix Expression

Example: $A + B * C$

Infix

 $A + B * C$
(usual form)

Postfix

 $A B C * +$
(operator after operands)

Prefix

 $+ A * B C$
(operator before operands)

5 Conversion Between Infix, Postfix and Prefix

Infix
 $A + B * C$

(using stack)

Postfix
 $A B C * +$

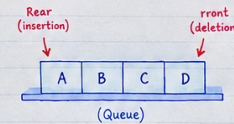
(using stack)

Prefix
 $+ A * B C$

- Use stack to handle operator precedence.
- Push operators to stack and pop when required.

6 Queue - Basic Idea

- A queue is a linear data structure where insertion is done at the rear and deletion is done at the front.
- It follows First In First Out (FIFO) principle.
- Think of it as a line of people, where the first one comes in, is the first one to go out.



FIFO
(First In
First Out)

7 Queue Operations

Enqueue(x)



Dequeue()



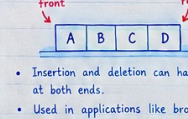
isEmpty()

True/False

isFull()

True/False

8 D-Queues (Double Ended Queue)



- Insertion and deletion can happen at both ends.
- Used in applications like browser history, dequeues, etc.

9 Priority Queue

P1 (highest)
P2
P3
P4 (lowest)

- Elements are arranged based on priority.
- Higher priority elements are served first.
- Implemented using heap or balanced tree.

Design by Sarafraz Navaz

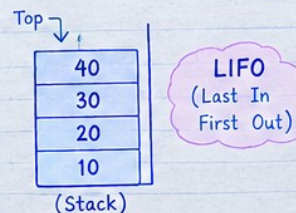
UNIT - II

STACKS AND QUEUES

1. Basic / Primitive Operations on Stack

Stack : A stack is a linear data structure in which insertion and deletion take place at the same end, called the top.

- It follows Last In First Out (LIFO) principle.
- You can only add or remove elements from the top.



1. PUSH(x)

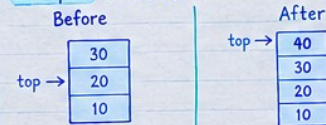
Purpose : Insert an element on top of the stack.

Condition : Stack must not be full.

- Steps :**
1. Check if $\text{top} == \text{MAX} - 1 \rightarrow$ Overflow
 2. $\text{top} = \text{top} + 1$
 3. $\text{stack}[\text{top}] = x$

If stack is full $\rightarrow$ Stack Overflow

Example : Push(40)



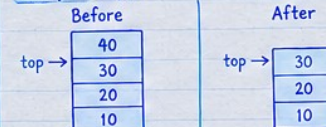
2. POP()

Purpose : Remove the top element from the stack.

Condition : Stack must not be empty.

- Steps :**
1. Check if $\text{top} == -1 \rightarrow$ Underflow
 2. $x = \text{stack}[\text{top}]$
 3. $\text{top} = \text{top} - 1$
 4. return x

Example : Pop()



If stack is empty $\rightarrow$ Stack Underflow

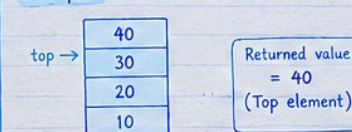
3. PEEK / TOP()

Purpose : Return the top element without removing it.

Condition : Stack must not be empty.

- Steps :**
1. Check if $\text{top} == -1 \rightarrow$ Underflow
 2. return $\text{stack}[\text{top}]$

Example : Peek()



4. isEmpty()

Purpose : Check whether the stack is empty.

Condition : $\text{top} == -1$ (if true $\rightarrow$ empty)

Return : 1 (true) or 0 (false)

5. isFull()

Purpose : Check whether the stack is full.

Condition : $\text{top} == \text{MAX} - 1$

Return : 1 (true) or 0 (false)

6. size()

Purpose : Return number of elements in stack.

Formula : $\text{size} = \text{top} + 1$

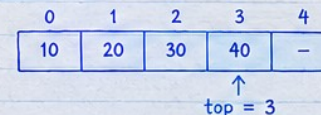
Design by Sarafraj Navaz

STACK IMPLEMENTATION IN C

1. Static (Array) Implementation

In static implementation, the stack is represented using an array of fixed size. The top points to the index of the last element.

Representation :

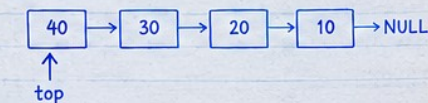


Limit : Size is fixed (e.g. $\text{MAX} = 5$).

2. Dynamic (Linked List) Implementation

In dynamic implementation, the stack is represented using a linked list. The top points to the first node.

Representation :



Advantage : No fixed size. Grows/shrinks as needed.

3. C Code for Stack Operations (Static Array Implementation)

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = -1;
```

Explanation :

- `#include <stdio.h>` $\rightarrow$ for `printf()`.
- `#define MAX 5` $\rightarrow$ maximum size of stack.
- `stack[]` $\rightarrow$ array to store elements.
- `top` $\rightarrow$ points to top element (-1 initially).

1. push(int x)

```
void push(int x) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = x;
    printf("Pushed: %d\n", x);
}
```

- Check if stack is full.
- If not, increment top and insert x .
- Show message.

2. pop()

```
int pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    int x = stack[top--];
    printf("Popped: %d\n", x);
    return x;
}
```

- Check if stack is empty.
- If not, get top element and decrement top.
- Return the value.

3. peek()

```
int peek() {
    if (top == -1) {
        printf("Stack is Empty\n");
        return -1;
    }
    return stack[top];
}
```

- Check if stack is empty.
- Return the top element without removing it.

4. isEmpty()

```
int isEmpty() {
    return (top == -1);
}
```

- Returns 1 if empty, else 0.

5. isFull()

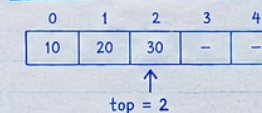
```
int isFull() {
    return (top == MAX - 1);
}
```

- Returns 1 if full, else 0.

Example Output :

Pushed: 10	Popped: 30
Pushed: 20	Top element is: 20
Pushed: 30	Stack is not empty
Top element is: 30	Stack is not full

Stack State (after 3 pushes):



Design by Sarafraj Navaz

STACK - BASIC OPERATIONS (C CODE)

Only the following operations are included:

push(), pop(), peek(), isFull(), isEmpty()

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = -1;
(Common setup for all functions)
```

1 push() Function

- Inserts an element into the stack.
- First checks if the stack is full.

```
void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return; // function yahin se exit ho jayega
    }
    top++; // top ko ek badhao
    stack[top] = value; // naya value store karo
    printf("%d pushed\n", value);
}
```

Example:

```
push(10); // [10]
push(20); // [10, 20]
push(30); // [10, 20, 30]
```

top →

30
20
10

Stack

2 pop() Function

- Removes an element from the top of the stack.
- First checks if the stack is empty.

```
int pop() {
    int item;
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1; // stack khali hai
    }
    item = stack[top];
    printf("%d popped\n", stack[top]);
    top--; // top ko ek ghatao
    return item;
}
```

Example:

```
pop(); // 30 popped
pop(); // 20 popped
pop(); // 10 popped
```

top →

20
10

Stack

3 peek() Function

- Returns the top element without removing it.
- First checks if the stack is empty.

```
int peek() {
    int item;
    if (top == -1) {
        printf("Stack is empty\n");
    }
    printf("Top element: %d\n", stack[top]);
    return item = stack[top];
}
```

Example:

```
peek(); // Top element: 30
(stack same rahega)
```

top →

30
20
10

Stack

4 isEmpty() Function

- Returns true (1) if the stack is empty, else false (0).

```
int isEmpty() {
    if (top == -1) {
        return 1; // TRUE - stack khali hai
    }
    return 0; // FALSE - stack me elements hain
}
```

Example:

```
if (isEmpty()) {
    printf("Stack is empty\n");
} else {
    printf("Stack is not empty\n");
}
```

(top = -1 → empty)

5 isFull() Function

- Returns true (1) if the stack is full, else false (0).

```
int isFull() {
    if (top == MAX - 1) {
        return 1; // TRUE - stack full hai
    }
    return 0; // FALSE - abhi jagah bachi hai
}
```

Example:

```
if (isFull()) {
    printf("Stack is full\n");
} else {
    printf("Stack is not full\n");
}
```

(top = MAX - 1 → full)

Design by
Sarafraj Navaz

1. FACTORIAL

1. Definition

- Factorial of a number n (denoted by $n!$) is the product of all positive integers from 1 to n .
- It is a classic example of recursion, because it can be defined in terms of a smaller subproblem.

Formula:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1, \quad n \geq 1$$

$$0! = 1$$

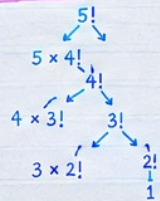
2. Recursive Approach

- Base case: $0! = 1$ and $1! = 1$
- Recursive relation: $n! = n \times (n-1)!$

3. Example ($n = 5$)

$$\begin{aligned} 5! &= 5 \times 4! \\ 4! &= 4 \times 3! \\ 3! &= 3 \times 2! \\ 2! &= 2 \times 1! \\ 1! &= 1 \end{aligned} \quad \rightarrow 120$$

Recursion Tree:



4. C Code

```
#include <stdio.h>
int fact(int n){
    if(n <= 1)
        return 1; // Base case
    return n * fact(n-1); // Recursive call
}
int main(){
    int n = 5;
    printf("Factorial of %d = %d", n, fact(n));
    return 0;
}
```

Output:

Factorial of 5 = 120

2. FIBONACCI SERIES

1. Definition

- The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding numbers.
- It starts with 0 and 1.

Formula:

$$F(n) = F(n-1) + F(n-2)$$

with $F(0) = 0, F(1) = 1$

2. Recursive Approach

- Base cases: $F(0) = 0, F(1) = 1$
- Recursive relation: $F(n) = F(n-1) + F(n-2)$

3. Example ($n = 6$)

$$\begin{aligned} F(6) &= F(5) + F(4) \\ &= 5 + 3 \\ &= 8 \end{aligned}$$

Fibonacci Series:

0, 1, 1, 2, 3, 5, 8, 13, ...

4. C Code

```
#include <stdio.h>
int fib(int n){
    if(n <= 1)
        return n; // Base case
    return fib(n-1) + fib(n-2); // Recursive call
}
int main(){
    int n = 6;
    printf("Fibonacci(%d) = %d", n, fib(n));
    return 0;
}
```

Output:

Fibonacci(6) = 8

3. TOWER OF HANOI

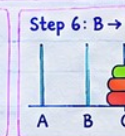
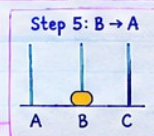
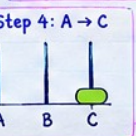
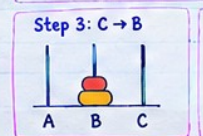
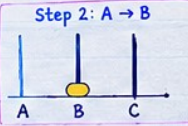
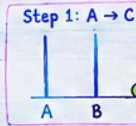
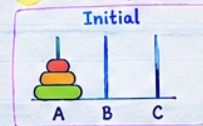
1. Problem Statement

- The Tower of Hanoi is a classic problem in recursion.
- It consists of three rods (A, B, C) and n disks of different sizes.
- The goal is to move all disks from the source rod to the destination rod, following these rules:
 - Only one disk can be moved at a time.
 - A larger disk cannot be placed on a smaller disk.
 - Only the top disk can be moved from a rod.

2. Recursive Approach

- Base case: $n = 1 \rightarrow$ move disk from A to C.
- Recursive case ($n > 1$):
 - Move $n - 1$ disks from A to B (using C).
 - Move the largest disk from A to C.
 - Move $n - 1$ disks from B to C (using A).

3. Example ($n = 3$)



4. C Code

```
#include <stdio.h>
void hanoi(int n, char from, char to, char aux){
    if(n == 1){
        printf("Move disk 1 from %c to %c\n", from, to);
        return;
    }
    hanoi(n-1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n-1, aux, to, from);
}
int main(){
    int n = 3;
    hanoi(n, 'A', 'C', 'B');
    return 0;
}
```

Output:

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C